

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)»

ОТЧЁТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
по теме:
РЕАЛИЗАЦИЯ ГЛОБАЛЬНОГО ВТОРИЧНОГО ИНДЕКСА И
СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЕГО ПРОИЗВОДИТЕЛЬНОСТИ С
ЛОКАЛЬНЫМИ ИНДЕКСАМИ В РАСПРЕДЕЛЕННОЙ СУБД PICODATA

Руководитель НИР,
Канд. техн. наук, доц.

подпись, дата

Романенков Д.В.

Исполнитель НИР,
Студент группы
М8О-403Б-22

подпись, дата

Клименко В.М.

РЕФЕРАТ

Отчёт 36 с., 1 рис., 2 табл., 15 ист.

РАСПРЕДЕЛЕННЫЕ СУБД, БАЗЫ ДАННЫХ, ВТОРИЧНЫЙ
ИНДЕКС, ГЛОБАЛЬНЫЙ ВТОРИЧНЫЙ ИНДЕКС, МАТЕМАТИЧЕСКОЕ
МОДЕЛИРОВАНИЕ, СИСТЕМЫ МАССОВОГО ОБСЛУЖИВАНИЯ

СОДЕРЖАНИЕ

Термины и определения	5
Перечень сокращений и обозначений	7
1 Вербальное описание области, направления работы, места и роли продукта, план работы	8
1.1 Системы управления базами данных	8
1.2 Системы массового обслуживания	8
1.3 Проблема поиска	9
1.4 Решение проблемы	9
1.5 Место и роль продукта НИР	10
2 Описание предметной области	12
2.1 Верхнеуровневое описание СУБД Picodata	12
2.1.1 Шардирование	13
2.2 Диаграмма «сущность-связь»	14
2.2.1 Описание условных обозначений	15
2.2.2 Описание сущностей	16
2.2.3 Описание связей	17
3 Математическая модель функционирования СУБД Picodata как СМО . .	19
3.1 Характеристика СУБД Picodata как СМО	19
3.1.1 Координатор	20
3.1.2 Исполнитель	22
3.1.3 Показатели эффективности	22
3.2 Поиск в локальном индексе	23
3.2.1 Расчет показателей эффективности	28
3.3 Поиск в глобальном индексе	28
3.4 Обновление в локальном индексе	28
3.5 Обновление в глобальном индексе	29

3.6	Результат расчетов и сравнение	29
4	Модель функционирования плагина распределенного индекса	30
4.1	Контекстная диаграмма	30
4.2	Диаграмма потоков данных	30
4.2.1	Функциональная декомпозиция	30
5	Архитектура плагина распределенного индекса	31
5.1	Архитектура плагина	31
5.1.1	Диаграмма развертывания	31
5.1.2	Представление данных	31
5.1.3	Описание алгоритмов	31
6	Описание способов определения показателей качества ПО	32
6.1	Показатели качества	32
6.2	Способы определения показателей качества	32
7	Программа и методики испытаний ПО	33
7.1	Модель функционирования системы тестирования	33
7.1.1	Контекстная диаграмма	33
7.1.2	Диаграмма потоков данных	33
7.2	Архитектура системы тестирования индексных структур	33
7.2.1	Общие сведения	33
7.2.2	Структурные представления	33
7.2.3	Архитектура решения	33
7.3	Сравнение полученных результатов с математическими моделями	33
	Заключение	34
	Список использованных источников	35

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями.

Инстанс — один запущенный процесс базы данных

Мастер — инстанс, принимающий запросы на запись и чтение

Реплика — инстанс, принимающий запросы только на чтение

Репликасет — набор инстансов, хранящих одинаковую часть данных, в котором один является мастером, а остальные являются репликами, принимающими и применяющими поток изменений от мастера

Шардирование — разбиение данных по репликасетам

Бакет — единица балансировки. Виртуальная сущность, к которой однозначно привязаны данные

Таблица — совокупность связанных данных, хранящихся в структурированном виде

Вторичный индекс — вспомогательная структура данных, используемая для ускорения поиска по указанным атрибутам

Первичный ключ — атрибуты записи, по которым идентифицируется запись таблицы

Вторичный ключ — атрибуты записи, по которым строится вторичный индекс

Ключ шардирования — атрибуты записи, по которым происходит определение ее бакета, чаще всего является первичным ключом таблицы

Шардированная таблица — таблица, данные которой распределены между репликасетами по бакетам

Кластер — набор репликасетов, предоставляющий доступ к единой СУБД

Плагин — единица программного обеспечения, представленная динамической библиотекой, расширяющая функционал инстанса

Драйвер — программное обеспечение, предоставляющее интерфейс взаимодействия с кластером

Персистентность — свойство данных сохраняться на диск, чтобы быть доступными после перезапуска инстанса

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения.

СУБД — система управления базами данных

ПО — программное обеспечение

ОЗУ — оперативное запоминающее устройство

ЦПУ — центральное процессорное устройство

ЛВИ — локальный вторичный индекс. Вторичный индекс, шардированный как и ключ шардирования индексируемой таблицы

ГВИ — глобальный вторичный индекс. Вторичный индекс, шардированный вне зависимости от ключа шардирования индексируемой таблицы

PK — primary key (первичный ключ)

FK — foreign key (вторичный ключ)

ER-диаграмма — entity-relationship (сущность-связь) диаграмма, выполненная в нотации Чена

Read-only нагрузка — модель обращения к данным, при которой происходит только чтение данных, без их изменения

Read-write нагрузка — модель обращения к данным, при которой происходят чтение и изменение данных

1 Вербальное описание области, направления работы, места и роли продукта, план работы

Для понимания прикладной области продукта НИР, нужно рассмотреть две сущности: системы управления базами данных (СУБД) и системы массового обслуживания (СМО).

1.1 Системы управления базами данных

Две единственные задачи СУБД — обновлять пользовательские данные и искать среди них определенные записи. В реляционных СУБД есть понятие первичного ключа списка хранимых структур (таблиц). Первичный ключ представляет из себя набор полей структур, по значению которого, запись в таблице определяется однозначно. В памяти подавляющего большинства СУБД поддерживается дополнительная структура данных, позволяющая ускорить поиск по нему — индекс.

Однако, часто возникает задача поиска записи по неидентифицирующим (вторичным) полям, что наталкивает на мысль использования подобной структуры данных, с единственным различием — необязательной однозначностью определения записи.

1.2 Системы массового обслуживания

Электронные СМО сталкиваются с вызовом эффективной обработки потока заявок, поступающих в случайные моменты времени. В ситуации, когда покупка более производительного аппаратного обеспечения (вертикальное масштабирование) одноканальной СМО становится слишком затратным или по-просту невозможным (ввиду законов физики и научного прогресса), переход на многоканальность ради увеличения скорости обработки потока заявок — единственный выход.

Распределенная реляционная СУБД Picodata является многоканальной СМО. Ее многоканальность проявляется распределением данных по множеству независимых серверов в зависимости от значения первичного ключа записи — шардированием.

1.3 Проблема поиска

Положим, что базовой задачей поиска в СУБД является задача поиска по значению первичного или вторичного ключа. Шардирование данных в СУБД Picodata позволяет оптимизировать задачу поиска по ключу шардирования (обычно он же является первичным ключом). Если:

- данные равномерно распределены по всем узлам кластера,
- количество данных в искомой таблице равно D ,
- количество узлов в кластере равно N ,
- ПО подключения к СУБД (драйвер) знает функцию, которая по значению первичного ключа определяет конкретный узел кластера, в котором должна храниться запись, называемой функцией шардирования,

то поиск в структуре данных, хранящей таблицу с искомой записью будет происходить среди $\frac{D}{N}$ записей, в то время, когда нераспределенным СУБД пришлось бы производить его среди полного набора записей D .

Однако, в СУБД Picodata нет решения задачи оптимизации поиска по вторичным ключам такой же эффективности. Сейчас происходит широкополосный опрос кластера с поиском в локальных индексах. То есть среди всех данных D , N серверов параллельно ищут запрашиваемую запись — каждая машина совершает поиск среди своей части $\frac{D}{N}$ записей.

Рассуждение выше наталкивает на мысль о том, что, например, если значение вторичного ключа так же однозначно определяет запись, как и первичный индекс (что нередкость), то количество совершённой бесполезной работы при подобном подходе увеличивается пропорционально количеству узлов кластера (N). Помимо этого, количество сетевых запросов растет пропорционально количеству узлов кластера, что тоже в высокой степени влияет на быстродействие поиска в СУБД.

1.4 Решение проблемы

Исходя из этого, было решено провести исследовательскую работу по изучению глобального вторичного индекса (ГВИ) в СУБД Picodata. Идея заключается в шардировании индексной структуры по значениям вторичного ключа индексируемой таблицы.

План работ следующий:

- 1) полно описать СУБД Picodata с предлагаемой индексной структурой и без как СМО;
- 2) построить математическую модель СМО (см [1]) для СУБД Picodata с предлагаемой индексной структурой и без: предоставить формулы расчета теоретических показателей эффективности СМО;
- 3) провести теоретический расчет показателей эффективности СМО;
- 4) разработать ПО, реализующее математическую модель в виде плагина к СУБД Picodata и тестирующее ПО для проведения экспериментов;
- 5) провести эксперименты для снятия показателей эффективности СМО на разработанном ПО при небольшом размере кластера, сравнить полученные результаты с теоретическим расчетом, объяснить расхождения, и сделать выводы — провести границы применимости распределенного индекса;
- 6) решить обратную задачу построения теоретических показателей эффективности — составить перечень рекомендаций по использованию распределенной индексной структуры.

1.5 Место и роль продукта НИР

Основным продуктом НИР является плагин к СУБД Picodata, добавляющий функционал глобальных индексов и интерфейс работы с ними.

Функциональные возможности плагина:

- создание глобального индекса;
- обновление индекса при изменении данных;
- удаление индекса;
- выполнение поисковых запросов с использованием глобального индекса.

Целевые пользователи плагина:

- администраторы СУБД Picodata;
- разработчики распределенных приложений.

Побочными продуктами НИР выступают:

- 1) математическая модель индексных структур СУБД Picodata;
- 2) показатели эффективности различных индексных структур;
- 3) методические рекомендации по выбору типа индекса;
- 4) драйвер использования плагина;
- 5) результаты нагрузочного тестирования.

2 Описание предметной области

Результатом данной главы является концептуальная схема предметной области, которая отражает основные сущности, их атрибуты и связи, существующие в контексте вторичных индексов в СУБД Picodata. Схема служит промежуточным звеном между вербальным описанием и последующим проектированием архитектуры программного продукта. Она позволяет:

- формализовать знания о предметной области;
- выявить ключевые объекты, участвующие в процессах индексирования и поиска;
- создать основу для разработки физической модели данных плагина.

2.1 Верхнеуровневое описание СУБД Picodata

Модель функционирования СУБД Picodata представлена текстовым верхнеуровневым описанием архитектуры, некоторых особенностей и внутренних процессов.

Picodata — это распределенная СУБД, основной фокус которой является отказоустойчивое хранение шардированных данных. Ее распределенность проявляется при соединении через сеть нескольких запущенных процессов Picodata (инстансов): они собираются в кластер — единую базу данных.

Каждый кластер состоит из репликasetов — набора инстансов, в которых один из инстансов (мастер) обрабатывает пользовательские запросы, а остальные являются его репликами. Разделение данных между репликасетами (шардирование) позволяет примерно равномерно распределять нагрузку среди всех репликasetов, например, при обновлении одной записи, обновляется только часть базы данных, расположенная на одном из инстансов.

Picodata представляет из себя многопоточное приложение. Главным потоком является `tx_thread` — из интересующего нас, в нем происходит доступ к данным и запускается код плагинов.

При использовании резидентного движка хранения `memtx`, обновления таблицы записываются в журнал упреждающей записи, а индексы хранятся в структуре данных B+*-дерево (смесь вариантов B+- и B*- деревьев) в

ОЗУ. При построении этого индекса, ключом становится конкатенация значения вторичного ключа с первичным, значит записи с совпадающими вторичными ключами идут подряд.

Среда потока доступа к данным исполнения является асинхронной, то есть используется кооперативная многозадачность, что позволяет максимально эффективно использовать ЦПУ во время ожидания ответа дискового хранилища или сетевого интерфейса.

2.1.1 Шардирование

Особо важно понимать механизм шардирования в СУБД Picodata. Равномерность распределения данных, а также предотвращение лишних сетевых запросов при изменении количества репликасетов достигается благодаря виртуальным сущностям — бакетам, которые однозначно идентифицируются положительным числом больше нуля и меньше заданного числа бакетов в кластере. Количество бакетов в кластере задается единожды при запуске первого инстанса и не изменяется впоследствии.

Среди репликасетов бакеты поделены поровну, например, если в кластере 3 репликасета, а бакетов 3000, то бакеты поделены по 1000 штук (например, отрезками), то есть:

- первому репликасету принадлежат бакеты с номерами от 1 по 1000;
- второму — от 1001 по 2000;
- третьему — от 2001 по 3000.

При создании шардированной таблицы, задается набор атрибутов, называемый ключом шардирования (обычно им же является первичный ключ) — это поля записи, на основе которых вычисляется бакет каждой записи. Номер бакета равен остатку от деления значения хеш-функции ключа шардирования.

Любые пользовательские запросы, пришедшие на инстанс, преобразовываются в план исполнения — набор операций, который должен исполнить движок хранения, чтобы ответить на запрос. Если для исполнения запроса должно быть задействовано несколько репликасетов, то на мастер каждого репликасета отправляется локальный план исполнения, который исполняется в локальном движке хранения, например, в случае memtx, это обход и обновление B+*-дерева в ОЗУ.

Про более подробное устройство индекса, построенного на B+*-дереве можно прочитать в [2–5].

Важно заметить, что и прием пользовательских запросов, и обработка локальных планов исполнения происходит в одном и том же потоке уровня операционной системы — `tx_thread`. Данное поведение возможно ввиду асинхронной среды исполнения этого потока.

При вставке, инстанс, принявший запрос, определяет в каком репликasetе должна быть запись по рассчитанному бакету, и отправляет в него локальный план вставки.

При поиске по ключу шардирования, инстанс, принявший запрос, определяет в каком репликasetе должна быть запись по рассчитанному бакету, и отправляет в него локальный план поиска в индексной структуре.

При поиске не по ключу шардирования, определить репликaset записи невозможно, поэтому происходит опрос всех репликasetов на наличие искомой записи (как раз этот случай и должен быть оптимизирован глобальным вторичным индексом).

TODO: приложить картинки примеров шардирования и sequence диаграмм алгоритмов?

Полную документацию СУБД Picodata можно прочитать на портале документации компании [6]. Подробнее про концепты распределенных систем можно прочитать в [7].

2.2 Диаграмма «сущность-связь»

Инфологическая схема предметной области представлена в виде диаграммы «сущность-связь» на рисунке 1.

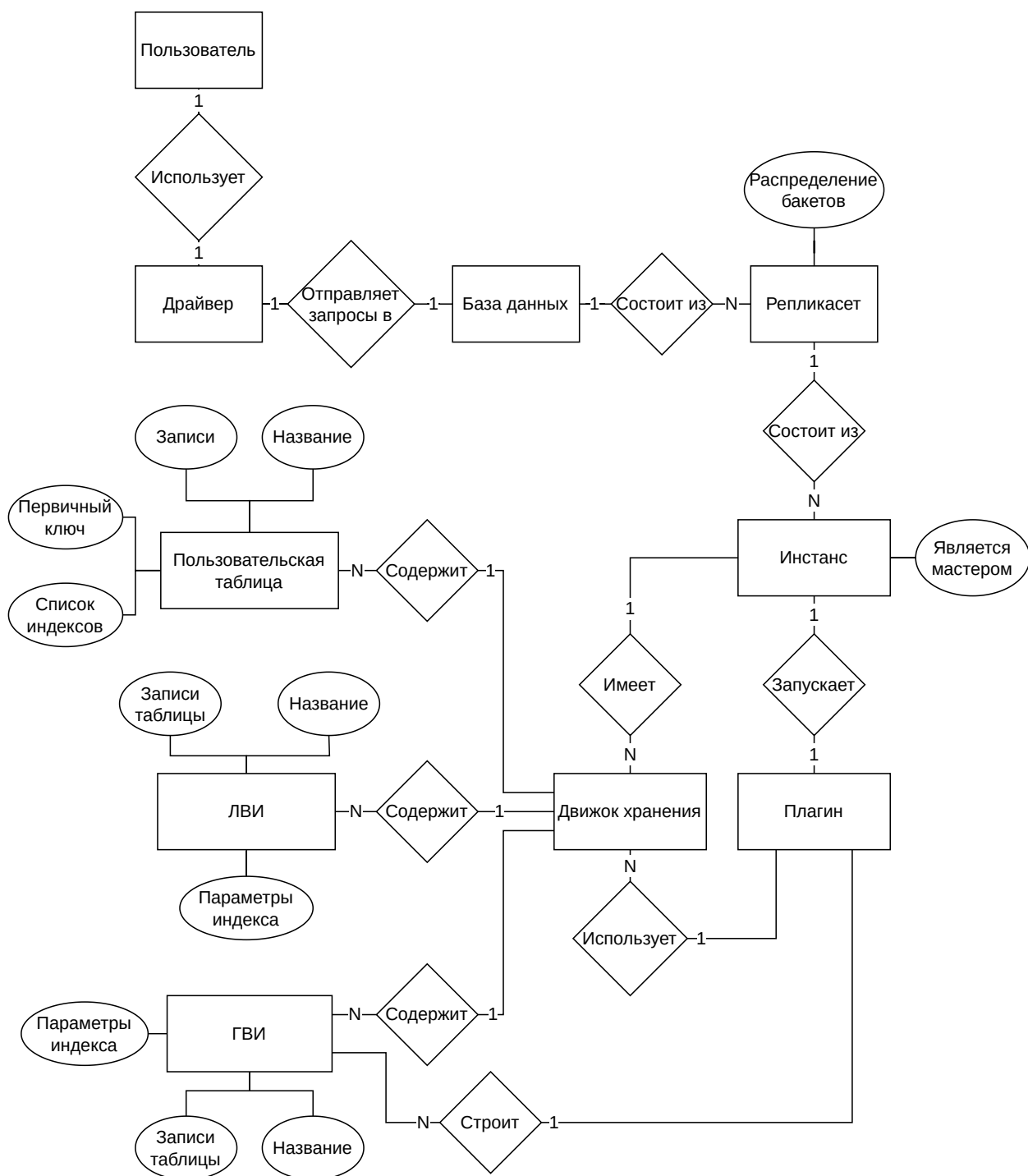


Рисунок 1 — ER-диаграмма предметной области продукта НИР

2.2.1 Описание условных обозначений

Диаграмма выполнена в нотации Чена для моделирования «сущность-связь». Используемые обозначения:

- прямоугольник — сущность;
- ромб — связь;
- овал — атрибут сущности;

- 1:1 — связь «один к одному»;
- 1:N — связь «один ко многим».

Нотация описана в [8].

2.2.2 Описание сущностей

В модели выделены следующие сущности и их атрибуты:

- пользователь — лицо, инициирующее запросы к СУБД;
- драйвер — ПО, предоставляющее интерфейс взаимодействия с СУБД;
- база данных — кластер СУБД Picodata;
- репликасеты — набор инстансов, хранящих одинаковую часть данных, в котором один является мастером, а остальные являются репликами, принимающими и применяющими поток изменений от мастера. Атрибуты:
 - распределение бакетов — список бакетов, принадлежащих данному репликасету;
- инстанс — один запущенный процесс Picodata. Атрибуты:
 - является мастером — булево значение, показывающее роль в репликасете: мастер или реплика;
- движок хранения — механизм хранения данных;
- пользовательская таблица — совокупность связанных данных о пользователях сервиса, хранящихся в структурированном виде. Атрибуты:
 - название — идентификатор таблицы;
 - первичный ключ — список идентификаторов полей, уникально определяющих запись;
 - записи — пользовательские данные;
 - список индексов — набор построенных над таблицей индексов;
- ЛВИ (локальный вторичный индекс) — индексная структура, шардированная как и индексируемая таблица. Атрибуты:
 - название — идентификатор индекса;

- записи таблицы — проиндексированные пользовательские данные;
- параметры индекса — значения настроек индекса: уникальность значений вторичных ключей;
- плагин — единица программного обеспечения, расширяющая функционал инстанса;
- ГВИ (глобальный вторичный индекс) — индексная структура, шардированная по индексируемым полям. Атрибуты:
 - название — идентификатор индекса;
 - записи таблицы — проиндексированные пользовательские данные;
 - параметры индекса — значения настроек индекса: уникальность значений вторичных ключей;

2.2.3 Описание связей

В модели выделены следующие связи между сущностями:

- **пользователь** → **драйвер**: пользователь использует драйвер для подключения к СУБД (1:1);
- **драйвер** → **база данных**: драйвер посылает запросы в базу данных (1:1);
- **база данных** → **репликасет**: база данных состоит из репликасетов (1:N);
- **репликасет** → **инстанс**: репликасет состоит из инстансов (1:N);
- **инстанс** → **движок хранения**: инстанс имеет несколько движков хранения (1:N);
- **движок хранения** → **пользовательская таблица**: движок хранения содержит множество пользовательских таблиц (1:N);
- **движок хранения** → **ЛВИ**: движок хранения содержит множество локальных вторичных индексов (1:N);
- **движок хранения** → **ГВИ**: движок хранения содержит множество глобальных вторичных индексов (1:N);
- **инстанс** → **плагин**: инстанс запускает плагин построения глобальных вторичных индексов (1:1);

- **плагин** → **движок хранения**: плагин использует движки хранения для построения глобальных вторичных индексов (1:N);
- **плагин** → **ГВИ**: плагин строит множество глобальных вторичных индексов (1:N).

3 Математическая модель функционирования СУБД Picodata как СМО

В данной главе изучаются индексные структуры распределенной СУБД Picodata при помощи построения математических моделей, основываясь на теории массового обслуживания. Рассматриваются локальный вторичный индекс (ЛВИ) — единственная индексная структура, реализованная в Picodata на данный момент, и новая, предлагаемая структура — глобальный вторичный индекс (ГВИ).

Результат главы предоставлен следующими сущностями:

- верхнеуровневое описание алгоритмов обновления каждой индексной структуры и поиска в них;
- вербальное описание и графическое представление каждой системы массового обслуживания (СМО);
- параметры эффективности каждой СМО;
- расчет параметров эффективности каждой СМО;

Все это необходимо для математического обоснования создания альтернативного алгоритма индексирования данных в распределенной СУБД Picodata.

В качестве материала для изучения теории массового обслуживания были использованы [9,10].

3.1 Характеристика СУБД Picodata как СМО

В контексте работы с индексами в СУБД Picodata, удобно разделить один инстанс на две СМО — на координатора и исполнителя. Важно понимать, что такое разделение на роли приведено сугубо для удобства построения математической модели, на деле же все инстансы являются одним и тем же приложением с идентичной логикой. Роль инстанса «зафиксирована» только на время обработки одного запроса и зависит исключительно от выбора пользователем инстанса, который примет запрос.

Работу координатора можно разделить на следующие последовательные пункты:

- 1) принять запрос пользователя;

- 2) разослать локальные планы исполнения на мастеров нужных репликасетов (исполнителей);
- 3) дождаться результата от исполнителей;
- 4) объединить полученные данные;
- 5) отправить пользователю запрашиваемые данные.

Работа исполнителя происходит между пунктами 2 и 3 работы координатора и разделяется на следующие последовательные действия:

- 1) принять локальный план исполнения от координатора;
- 2) исполнить план с помощью движка хранения на своей части данных;
- 3) отправить результат исполнения обратно на координатора.

Так можно увидеть, что СМО, которой описывается координатор является системой более высокого уровня, по сравнению с СМО, описывающей исполнителя. Координатор «управляет» всем кластером на время запроса, а исполнитель — только локальными структурами данных. Получается, один координатор бьет изначальный запрос на подзадачи, которые решают несколько исполнителей.

3.1.1 Координатор

В таблице 1 приведена характеристика координатора как СМО. Эти положения верны для обеих индексных структур и являются справедливыми и для поиска, и для обновления:

Таблица 1 — Характеристика роли координатора инстанса в СУБД Picodata как СМО

Элемент СМО	Элемент СУБД
Канал обслуживания	tx_thread каждого инстанса
Поток заявок	Запросы на поиск или обновление данных шардированной таблицы
Поток ответов	Отфильтрованные данные таблицы в случае поиска и подтверждение операции в случае обновления таблицы
Дисциплина очереди	Очередь бесконечной длины с ограничением по времени ожидания (таймаут заявки)

Элемент СМО	Элемент СУБД
Количество фаз	Несколько — один запрос обрабатывается несколькими каналами обслуживания

Небольшая пометка про количество фаз: обработка заявки происходит одним координатором и несколькими исполнителями, так как канал обслуживания у них общий, то и количество фаз решено поставить больше единицы.

Для простоты модели, допустим, что в течение работы количество заявок является случайной величиной следующей закону распределения Пуассона (время между заявками следует экспоненциальному закону распределения) с параметром λ , причем этот поток обладает свойствами:

- стационарности — число заявок не зависит от времени начала работы СМО;
- ординарности — в небольшой промежуток времени мал шанс получения нескольких заявок;
- является потоком без последствия — количество заявок в настоящем не зависит от количества заявок в прошлом.

Более подробное описание этих свойств можно найти в [9].

Также, чтобы избежать излишней сложности модели, постановим, что каждый репликасет кластера состоит из одного инстанса. Это не влияет на точность модели, так как реплики являются резервным хранилищем, на котором не исполняются запросы.

Для расчета задержки при сетевых запросах, используем случайную величину с гамма-распределением согласно исследованиям [11,12].

В контексте поиска по индексу, элементарной задачей является нахождение записей с фильтрацией по значению одного вторичного ключа.

В контексте обновления индекса, элементарной задачей является обновление одной записи во вторичном индексе.

Именно алгоритм координирования кластера, зависящий от вида индексной структуры и типа запроса, по большей части определяет время обслуживания заявки.

В пунктах 3.2, 3.3, 3.4 и 3.5 описаны алгоритмы координирования кластера при поиске и обновлении как СМО для локального и глобального индекса.

3.1.2 Исполнитель

В таблице 2 приведена характеристика исполнителя как СМО. Эти положения верны для обеих индексных структур и являются справедливыми и для поиска, и для обновления:

Таблица 2 — Характеристика роли координатора инстанса в СУБД Picodata как СМО

Элемент СМО	Элемент СУБД
Канал обслуживания	tx_thread данного инстанса
Поток заявок	Локальные планы исполнения поиска или обновления
Поток ответов	Отфильтрованные данные части таблицы репликасета в случае поиска, или подтверждение успешной операции в случае обновления таблицы
Дисциплина очереди	Очередь бесконечной длины с ограничением по времени ожидания (таймаут заявки)
Количество фаз	Одна — всего один обслуживающий канал

Для расчета времени работы будем ориентироваться на количество операций сравнения ключей при поиске и обновлении в B^{+} -дереве, деленное на частоту проведения этих операций ЦПУ и количество операций загрузки данных по указателям, деленное на частоту загрузки блока памяти из запоминающего устройства.

Также, будем учитывать время, потраченное на запись в журнал упреждающей записи, и скажем, что оно константное, так как представляет из себя просто добавление байтов в конец файла, указатель на конец которого известен в момент записи.

3.1.3 Показатели эффективности

Возьмем следующие показатели эффективности СМО:

- максимальное количество запросов в секунду;

- вероятность таймаута заявки;
- среднее время ожидания обслуживания;
- среднее число занятых инстансов.

Для построения расчетов показателей эффективности используются следующие параметры:

- количество репликасетов в кластере N ;
- объем пользовательских данных D (количество записей в таблице);
- количество уникальных вторичных ключей (кардинальность множества вторичных ключей) K ;
- размер одной записи r (в байтах);
- частота операций сравнения в секунду на ЦПУ f_{cpu} ;
- частота операций загрузки и записи данных по указателю из устройства памяти f_{mem} ;
- порядок B+*-дерева m ;
- время таймаута T_{timeout} в секундах;
- скорость сети между пользователем и кластером СУБД v_{user} в байтах в секунду;
- скорость сети между инстансами v_{cluster} в байтах в секунду;
- значения параметров случайных величин:
 - λ_{search} и λ_{update} — интенсивности потоков заявок (заявок в секунду) на поиск и обновление соответственно;
 - k_{user} и θ_{user} — параметры гамма-распределения времени сетевой задержки между пользователем и кластером СУБД;
 - k_{cluster} и θ_{cluster} — параметры гамма-распределения времени сетевой задержки между инстансами СУБД.

3.2 Поиск в локальном индексе

Алгоритм поиска в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, составляет план локального поиска и отправляет его всем исполнителем (мастеру каждого репликасета);

- 2) на каждом исполнителе происходит поиск записей по значению вторичного ключа в B+*-дереве части индекса;
- 3) исполнитель отправляет найденные записи координатору;
- 4) координатор ждет ответов от всех репликасетов, объединяет их и отправляет найденные записи пользователю.

Каждая заявка на поиск в координатор порождает N заявок исполнителям, причем интенсивность потока исполнителя так же равна λ_{search} . Интенсивность потока во всем кластере суммарно равна $\lambda_{\text{search}} N$.

Время ожидания пользователя ($T_{\text{search}}^{\text{user}}$) при отправлении заявки поиска является суммой следующих значений:

- 1) времени задержки сети пользовательского запроса $t_{\text{user}}^{\text{request}} \sim \Gamma(k_{\text{user}}, \theta_{\text{user}})$;
- 2) времени ожидания обработки запроса координатором $W_{\text{search}}^{\text{coordinate}}$;
- 3) времени задержки сети отправления результата пользователю $t_{\text{user}}^{\text{response}} \sim \Gamma(k_{\text{user}}, \theta_{\text{user}})$ и времени передачи всех записей по сети, зависящей от количества найденных данных S , размера одной записи r и скорости передачи данных между кластером и пользователем v_{user} .

Время передачи запроса не учитывается в модели, так как занимает мало времени, по сравнению с другими величинами.

В математической нотации это записывается следующим образом:

$$T_{\text{search}}^{\text{user}} = t_{\text{user}}^{\text{request}} + W_{\text{search}}^{\text{coordinate}} + t_{\text{user}}^{\text{response}} + \frac{Sr}{v_{\text{user}}}. \quad (1)$$

Время работы координатора $T_{\text{search}}^{\text{coordinate}}$, необходимое для расчета времени ожидания обработки запроса координатором $W_{\text{search}}^{\text{coordinate}}$, является максимальным значением суммы следующих значений среди всех репликасетов ($i = \overline{1..N}$):

- 1) задержка сети отправления локального плана на исполнителя $t_{\text{replicaset}}^{\text{request}, i} \sim \Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$;
- 2) времени ожидания обработки подзадачи в исполнителе $W_{\text{search}}^{\text{execute}}$;
- 3) задержка сети отправления результата координатору $t_{\text{replicaset}}^{\text{response}, i} \sim \Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ и времени передачи всех записей по сети,

зависящей от количества найденных данных s на репликasete, размера одной записи r и скорости передачи сети в кластере v_{cluster} .

В математической нотации это записывается следующим образом:

$$T_{\text{search}}^{\text{coordinate}} = \max_{i=1..N} \left[t_{\text{replicaset}}^{\text{request},i} + W_{\text{search}}^{\text{execute}} + t_{\text{replicaset}}^{\text{response},i} + \frac{sr}{v_{\text{cluster}}} \right]. \quad (2)$$

Время составления и передачи плана исполнения и время объединения результатов по сравнению с другими величинами занимают мало времени, поэтому не учитываются в модели.

Все параметры, необходимые для расчета, кроме количества данных во всем кластере S и на каждом репликasete s и времени ожидания обработки подзадачи исполнителем $W_{\text{search}}^{\text{execute}}$ известны. Вычислим неизвестные параметры.

На каждом исполнителе хранится по $d = \frac{D}{N}$ записей, так что поиск первой записи по вторичному ключу в B^+ -дереве требует $\log_m d$ операций перехода по указателям и $\log_m d \cdot \log_2(2m - 1)$ операций сравнения ключей в худшем случае.

При равномерном распределении записей по K вторичным ключам, ожидаемое число подходящих записей на одном репликasete будет равно $s = \frac{d}{K}$, а во всем кластере — $S = sN = \frac{D}{K}$. Если рассмотреть случай, когда искомый ключ является самым правым ключом листа, при этом все листья правее имеют $\frac{2}{3}(2m - 1)$ ключей (минимальное количество ключей в узле в соответствии с определением B^+ -деревя), получаем, что чтение s записей из листовых узлов B^+ -деревя требует максимум $\frac{s-1}{\frac{2}{3}(2m-1)}$ дополнительных переходов по указателям между листьями. Итого, время решения подзадачи поиска равно

$$T_{\text{search}}^{\text{execute}} = \frac{\log_m d \cdot \log_2(2m - 1)}{f_{\text{cpu}}} + \frac{\log_m d + 3 \cdot \frac{s-1}{4m-2}}{f_{\text{mem}}}. \quad (3)$$

Важно заметить, что это детерминированная, а не случайная величина, так как рассматривается худший случай при поиске в B^+ -дереве. Ее дисперсия равна нулю, а математическое ожидание равно себе самой.

Загрузка исполнителя равна:

$$\rho_{\text{search}}^{\text{executor}} = \lambda_{\text{search}} \cdot T_{\text{search}}^{\text{execute}}, \quad (4)$$

Исполнитель будет справляться с нагрузкой при $\rho_{\text{search}}^{\text{executor}} < 1$.

По формуле Полячека—Хинчина [10] рассчитаем среднее количество заявок в исполнителе (количество заявок в очереди и ныне обслуживающихся):

$$\begin{aligned} L_{\text{search}}^{\text{execute}} &= \rho_{\text{search}}^{\text{executor}} + \frac{\rho_{\text{search}}^{\text{executor}^2} + \lambda_{\text{search}} D[T_{\text{search}}^{\text{execute}}]}{2(1 - \rho_{\text{search}}^{\text{executor}})} \\ &= \rho_{\text{search}}^{\text{executor}} + \frac{\rho_{\text{search}}^{\text{executor}^2}}{2(1 - \rho_{\text{search}}^{\text{executor}})}, \end{aligned} \quad (5)$$

и среднее время ожидания заявки в исполнителе в соответствии с законом Литтла, приведенным в [9,10]:

$$W_{\text{search}}^{\text{execute}} = \frac{L_{\text{search}}^{\text{execute}}}{\lambda_{\text{search}}} = T_{\text{search}}^{\text{execute}} + \frac{\rho_{\text{search}}^{\text{executor}} T_{\text{search}}^{\text{execute}}}{2(1 - \rho_{\text{search}}^{\text{executor}})} \quad (6)$$

Оценим время работы координатора, связанное с одним репликasetом:

$$T_{\text{search},i}^{\text{coordinate}} = t_{\text{replicaset}}^{\text{request},i} + W_{\text{search}}^{\text{execute}} + t_{\text{replicaset}}^{\text{response},i} + \frac{sr}{v_{\text{cluster}}}. \quad (7)$$

Только времена сетевой задержки ($t_{\text{replicaset}}^{\text{request},i}$, $t_{\text{replicaset}}^{\text{response},i}$) — случайные величины, остальные — константы, так что, согласно [13], время работы координатора на одном репликasetе — случайная величина, имеющая сдвинутое гамма-распределение. Случайную часть $T_{\text{search},i}^{\text{coordinate}}$ обозначим Y , а неслучайную — $C_{\text{coordinate}}$:

$$C_{\text{coordinate}} = W_{\text{search}}^{\text{execute}} + \frac{sr}{v_{\text{cluster}}}, \quad (8)$$

$$Y = T_{\text{search},i}^{\text{coordinate}} - C_{\text{coordinate}} \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}}). \quad (9)$$

Следовательно функция распределения равна:

$$F_Y(t) = P(Y \leq t) = \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})}. \quad (10)$$

Согласно формуле 2, время работы координатора равно:

$$T_{\text{search}}^{\text{coordinate}} = \max_{i=1..N} T_{\text{search},i}^{\text{coordinate}}, \quad (11)$$

случайную часть $T_{\text{search}}^{\text{coordinate}}$ обозначим Z , максимум среди одинаковых неслучайных величин $C_{\text{coordinate}}$ равен самому себе, поэтому:

$$Z = T_{\text{search}}^{\text{coordinate}} - C_{\text{coordinate}} \quad (12)$$

значит, функция распределения Z равна:

$$F_Z(t) = [F_Y(t)]^N = \left[\frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N, \quad (13)$$

что позволяет нам численно найти математическое ожидание и дисперсию $T_{\text{search}}^{\text{coordinate}}$ через функцию выживания [14]:

$$M[Z] = \int_0^\infty \left(1 - \left[\frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt, \quad (14)$$

$$M[Z^2] = \int_0^\infty 2t \left(1 - \left[\frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt, \quad (15)$$

$$M[T_{\text{search}}^{\text{coordinate}}] = M[Z] + C_{\text{coordinate}}, \quad (16)$$

$$D[T_{\text{search}}^{\text{coordinate}}] = D[Z] = M[Z^2] - (M[Z])^2. \quad (17)$$

TODO: распределение W через Клейнрока 5.7

Вновь используя формулу Полячека—Хинчина, найдем среднее количество заявок, теперь уже в координаторе ($W_{\text{search}}^{\text{coordinate}}$):

$$\rho_{\text{search}}^{\text{coordinator}} = \lambda_{\text{search}} \cdot T_{\text{search}}^{\text{coordinate}} \quad (18)$$

$$M[L_{\text{search}}^{\text{coordinate}}] = \rho_{\text{search}}^{\text{coordinator}} + \frac{\rho_{\text{search}}^{\text{coordinator}^2} + \lambda_{\text{search}}^2 \cdot D[T_{\text{search}}^{\text{coordinate}}]}{2(1 - \rho_{\text{search}}^{\text{coordinator}})} \quad (19)$$

$$\begin{aligned}
M[W_{\text{search}}^{\text{coordinate}}] &= \frac{L_{\text{search}}^{\text{coordinate}}}{\lambda_{\text{search}}} = \\
&= M[T_{\text{search}}^{\text{coordinate}}] + \frac{\rho_{\text{search}}^{\text{coordinator}} M[T_{\text{search}}^{\text{coordinate}}] + \lambda_{\text{search}} D[T_{\text{search}}^{\text{coordinate}}]}{2(1 - \rho_{\text{search}}^{\text{coordinator}})}
\end{aligned} \tag{20}$$

Для времени ожидания пользователя $T_{\text{search}}^{\text{user}}$, определенной в формуле 1 случайную величину обозначим V , тогда:

$$V = t_{\text{user}}^{\text{request}} + t_{\text{user}}^{\text{response}} \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}}), \tag{21}$$

из чего можно найти математическое ожидание и дисперсию времени ожидания пользователя:

$$\begin{aligned}
M[T_{\text{search}}^{\text{user}}] &= M[V] + M[W_{\text{search}}^{\text{coordinate}}] + \frac{Sr}{v_{\text{user}}} = \\
&= k_{\text{user}} \theta_{\text{user}} + M[W_{\text{search}}^{\text{coordinate}}] + \frac{Sr}{v_{\text{user}}},
\end{aligned} \tag{22}$$

$$D[T_{\text{search}}^{\text{user}}] = D[V] + D[W_{\text{search}}^{\text{coordinate}}]. \tag{23}$$

3.2.1 Расчет показателей эффективностей

3.3 Поиск в глобальном индексе

3.4 Обновление в локальном индексе

Алгоритм записи в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, рассчитывает бакет обновляемой записи и отправляет локальный план исполнения соответствующему исполнителю;
- 2) исполнитель добавляет информацию об обновлении таблицы в журнал упреждающей записи, обновляет $B+^*$ -дерево индекса первичного ключа и обновляет $B+^*$ -дерево вторичного индекса;
- 3) исполнитель отправляет подтверждение записи координатору;
- 4) координатор отправляет подтверждение записи пользователю.

Получается, каждая заявка на обновление порождает ровно одну подзадачу для одного исполнителя. Поток запросов координатора λ_{update} распределяется равномерно на исполнителей: $\lambda_{\text{update}}^{(\text{sub})} = \frac{\lambda_{\text{update}}}{N}$.

Обновление начинается с записи в журнал упреждающей записи, которое занимает константное время T_{WAL} . После этого происходит поиск позиции ключа в B⁺-дереве для обновления, на что тратится $\log_m d \cdot \log_2(2m - 1)$ операций сравнения ключей и $\log_m d$ переходов по указателям.

После поиска происходит обновление дерева, например вставка нового элемента. В лучшем случае, когда лист не заполнен до конца, вставка происходит за одну операцию записи в запоминающее устройство, в худшем — все узлы в пути до листа (включая сам лист) нужно разделить на два. Для простоты модели, сошлемся на статью [15], в которой доказано, что средняя заполненность B^{*}-дерева, а значит и средняя заполненность каждого узла равна $P_{\text{full}} = 0.81$, где 0 — узел пуст, 1 — узел полон. Получается, что в среднем остается $(2m - 1)(1 - 0.81) = (2m - 1)0.19$ вставок перед переполнением узла, соответственно вероятность переполнения на конкретной вставке равна $((2m - 1)0.19)^{-1} \approx \frac{5.26}{2m-1}$. Разделение узла происходит за 3 операции записи — перезапись разделенного узла, запись нового братского узла и перезапись родительского узла. Случай каскадного разделения родительских узлов возможен, но маловероятен (вероятность этого события уменьшается с каждым родителем экспоненциально), так что он не учтен.

Итого, время решения подзадачи обновления равно:

$$T_{\text{update}}^{\text{execute}} = T_{\text{WAL}} + \frac{\log_m d \cdot \log_2(2m - 1)}{f_{\text{cpu}}} + \frac{\log_m d + 3 \cdot \frac{5.26}{2m-1}}{f_{\text{mem}}} \quad (24)$$

Это тоже детерминированная величина.

3.5 Обновление в глобальном индексе

3.6 Результат расчетов и сравнение

4 Модель функционирования плагина распределенного индекса

4.1 Контекстная диаграмма

4.2 Диаграмма потоков данных

4.2.1 Функциональная декомпозиция

5 Архитектура плагина распределенного индекса

5.1 Архитектура плагина

5.1.1 Диаграмма развертывания

5.1.2 Представление данных

5.1.3 Описание алгоритмов

6 Описание способов определения показателей качества ПО

6.1 Показатели качества

6.2 Способы определения показателей качества

7 Программа и методики испытаний ПО

7.1 Модель функционирования системы тестирования

7.1.1 Контекстная диаграмма

7.1.2 Диаграмма потоков данных

7.1.2.1 Функциональная декомпозиция

7.2 Архитектура системы тестирования индексных структур

7.2.1 Общие сведения

7.2.2 Структурные представления

7.2.3 Архитектура решения

7.2.3.1 Основные положения архитектуры

7.2.3.2 Диаграмма развертывания

7.2.3.3 Представление данных

7.3 Сравнение полученных результатов с математическими моделями

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Вентцель Е. С. Исследование операций: задачи, принципы, методология. 2-е изд. "Наука" Главная редакция физико-математической литературы, 1988.
2. Википедия. В+-дерево [Электронный ресурс]. 2026. URL: <https://ru.wikipedia.org/wiki/B%E2%81%BA-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE>.
3. Википедия. В*-дерево [Электронный ресурс]. 2026. URL: https://ru.wikipedia.org/wiki/B*-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE.
4. Tarantool Developers. Tarantool source code: bps_tree.h [Электронный ресурс]. 2025. URL: https://github.com/tarantool/tarantool/blob/4e27591f685e9c695853a0165d4081148d12e315/src/lib/salad/bps_tree.h (дата обращения: 11.04.2026).
5. Бабин О. Как написать свой индекс в Tarantool [Электронный ресурс]. 2020. URL: <https://habr.com/ru/companies/vk/articles/505880/>.
6. Портал документации Picodata [Электронный ресурс]. ООО "Пикодата", 2026. URL: <https://docs.picodata.io/picodata/stable/> (дата обращения: 02.03.2026).
7. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA: O'Reilly Media, 2017.
8. Chen P. P.-S. The entity-relationship model—toward a unified view of data // ACM Transactions on Database Systems. Association for Computing Machinery (ACM), 1976. Т. 1, № 1. Сс. 9–36.
9. Розенберг В. Я., Прохоров А. И. Что такое теория массового обслуживания. 2-е изд. Москва: Советское радио, 1965. С. 256.
10. Shortle J. F. и др. Fundamentals of Queueing Theory. 5-е изд. Hoboken, NJ: Wiley, 2018.
11. Liu Y., Li J., Gu N. Modeling Internet Link Delay Based on Measurement // 2009 International Conference on Computational Science and Engineering. Vancouver, BC, Canada: IEEE, 2009. Т. 4. Сс. 874–879.

12. Valadão E. D. и др. Caracterização de tempos de ida-e-volta na Internet // Revista de Informática Teórica e Aplicada. Porto Alegre, Brazil: UFRGS, 2012. Т. 19, № 2. Сс. 4–20.
13. Кибзун А. И., Горяинова Е. Р., Наумов А. В. Теория вероятностей и математическая статистика. Базовый курс с примерами и задачами. 3-е изд. Москва: Физматлит, 2011.
14. Ross S. M. A First Course in Probability. 10-е изд. Pearson, 2019.
15. Leung C. H. C. Approximate storage utilisation of B-trees: A simple derivation and generalisations // Information Processing Letters. Elsevier, 1984. Т. 19, № 4. Сс. 199–201.